

OCCUPATIONAL CERTIFICATE

SOFTWARE DEVELOPER

SAQA QUAL ID: 118707 | NQF LEVEL 5

Design an E-Commerce Marketplace and Order Fulfilment System

A design-first practical assignment for an online retail platform

Learner name	
Student number	
Facilitator	
Training provider	
Submission date	
Group members	

Assignment type: Analysis and design practical | Suggested duration: 4 weeks | Total: 100 marks

Use only sandbox payment services and fictional customer data. No real card numbers, identity numbers or confidential business data may be used.

Document Purpose and Qualification Alignment

This practical assignment guides learners through the analysis and design stages of a realistic software project. It supports the Occupational Certificate: Software Developer, SAQA Qualification ID 118707, by requiring learners to interpret a business problem, gather and refine requirements, model processes and data, design interfaces and architecture, plan security and testing, and present a traceable solution before programming begins.

Assignment information	Detail
Project environment	Mzansi Market Online
Phase	Analysis and design
Suggested duration	4 weeks
Mode	Individual or groups of 3-5 learners
Total marks	100
Primary output	Software design portfolio and presentation

Learning Outcomes

- Analyse a stakeholder problem and define an appropriate project scope.
- Document functional, non-functional and business requirements.
- Create user stories, acceptance criteria and a prioritised product backlog.
- Model business processes using flowcharts, swimlanes or UML activity diagrams.
- Create use-case, class, sequence and data models that are consistent with the requirements.
- Design accessible user interfaces, navigation and validation behaviour.
- Propose algorithms, API operations and a suitable high-level architecture.
- Identify security, privacy, ethical, operational and data-quality risks.
- Develop test cases and a requirements traceability matrix.
- Communicate and defend design decisions professionally.

Design-first rule: Students must not begin full implementation until the facilitator has reviewed the requirements, models, database design and wireframes.

Project Scenario

Background

Mzansi Market Online is a fictional growing retailer that sells locally produced household goods, clothing, electronics accessories and stationery. Orders are currently captured through social media, telephone calls and spreadsheets. Product, stock, customer and delivery information is duplicated, and staff struggle to track payments, fulfil orders and manage returns.

Current Challenges

- Customers cannot reliably see current product availability.
- Orders from different channels are captured inconsistently.
- Stock may be promised to more than one customer.
- Order totals, discounts and delivery charges are calculated manually.
- Payment status and order status are not centrally tracked.
- Managers lack dependable sales, stock and fulfilment reports.
- Returns and refunds are not linked to the original order.
- There is no complete audit history of important changes.

Central Project Question

Question: How can a secure, accessible and responsive e-commerce solution improve product discovery, ordering, payment-status tracking, fulfilment and after-sales service?

Proposed Users and Responsibilities

User or role	Main responsibilities
Customer	Register, browse products, manage a cart, place an order, view order history and request a return.
Product Administrator	Maintain product, category, price, image metadata, promotion and stock information.
Order Fulfilment Employee	Review paid orders, reserve stock, pick and pack items, and update fulfilment status.
Business Manager	View reports, approve selected refunds, monitor sales, stock and service performance.
System Administrator	Manage users, roles, configuration and audit access without changing business transactions.

Core Business Processes

1. Customer registration and authentication
2. Product search, filtering and product-detail viewing
3. Shopping-cart management and checkout
4. Payment-status recording through a simulated or sandbox gateway
5. Stock reservation, picking, packing and dispatch
6. Order cancellation, return and refund request handling
7. Promotion and discount management
8. Sales, stock and fulfilment reporting

Assignment Instructions and Deliverables

You have been appointed as a junior software development team. Analyse the scenario and prepare a complete design proposal that another team could use as the basis for implementation. Clearly label assumptions. Use fictional, non-sensitive data and comply with the project-specific safety note on the cover page.

Deliverable 1: Project Charter

Prepare a concise project charter containing:

- Project name and background
- Problem statement
- Purpose and measurable objectives
- Expected benefits
- In-scope and out-of-scope items
- Assumptions and constraints
- Major risks and dependencies
- Stakeholder list
- Success criteria

Charter field	Learner response
Problem statement	
Purpose	
Objective 1	
Objective 2	
In scope	
Out of scope	
Key assumption	
Key constraint	
Success measure	

Deliverable 2: Stakeholder Analysis and Requirements-Gathering Plan

Identify at least five stakeholders and use at least three requirements-gathering techniques, such as interviews, workshops, observation, document analysis, questionnaires or prototyping.

Stakeholder	Interest or need	Influence	Engagement method

Prepare at least ten scenario-specific interview or workshop questions. Questions must discover current problems, rules, exceptions, reporting needs, access restrictions and success measures.

Deliverable 3: Software Requirements Specification

Create an SRS containing an introduction, system context, assumptions, functional requirements, non-functional requirements, business rules, data requirements, external interfaces and acceptance constraints.

Example Functional Requirements

ID	Example requirement	Priority
FR-001	The system shall allow a customer to register and authenticate using approved credentials.	Must
FR-002	The system shall allow users to search and filter active products by name, category, price range and availability.	Must
FR-003	The system shall maintain a shopping cart for an authenticated customer.	Must
FR-004	The system shall calculate item totals, discounts, delivery charges and the final order total.	Must
FR-005	The system shall prevent checkout when the requested quantity exceeds available stock.	Must
FR-006	The system shall record payment status without storing sensitive payment-card data.	Must
FR-007	The system shall allow authorised employees to progress an order through approved fulfilment statuses.	Must
FR-008	The system shall allow a customer to request a return against an eligible completed order.	Should

Expand the list to at least 25 functional requirements. Each requirement must be testable, unambiguous and assigned a unique ID.

Required Non-Functional Categories

- Security and privacy
- Performance and capacity
- Availability and recovery
- Usability and accessibility
- Reliability and data integrity
- Maintainability and supportability
- Compatibility and responsive behaviour
- Auditability and monitoring

Scenario Business Rules

- A product must belong to at least one active category.
- A product code or SKU must be unique.
- A cart item quantity must be greater than zero.
- An order must contain at least one order item.
- The order total must be calculated from server-side product prices and approved discounts.
- Stock must not be reduced below zero.
- Payment-card numbers must not be stored in the learning solution.
- A refund request must reference an original order and reason.
- Only authorised roles may change prices or stock balances.
- Completed and refunded transactions must remain available for audit and reporting.

Add at least five additional business rules discovered or assumed by the team. Clearly label assumptions that require stakeholder validation.

Deliverable 4: User Stories, Acceptance Criteria and Product Backlog

Write at least 18 user stories in the format: As a [role], I want [action] so that [benefit]. Provide Given/When/Then acceptance criteria and trace each story to one or more requirements.

ID	Role	Action	Benefit
US-001	Customer	browse available products by category	I can quickly find products that interest me
US-002	Customer	add, update and remove products in my cart	I can prepare an order before checkout
US-003	Customer	see the full order cost before submission	I understand what I will pay
US-004	Customer	track the status of my order	I do not need to contact support for routine updates
US-005	Product Administrator	maintain products, prices and stock alerts	customers receive accurate information
US-006	Fulfilment Employee	view paid orders that require preparation	I can process work in the correct sequence
US-007	Business Manager	view sales and stock reports	I can make evidence-based decisions
US-008	Customer	request a return for an eligible item	after-sales service is traceable

Create a prioritised backlog of at least 25 items using Must have, Should have, Could have and Deferred. Include estimated effort and a proposed iteration.

Backlog ID	Feature	Priority	Effort	Iteration

Deliverable 5: Business Process Models

Create a current-state model and at least three proposed-state models. Use clear start/end events, activities, decisions, responsible roles, data inputs, outputs and exception paths.

- Current-state process for Mzansi Market Online
- Proposed process: Customer registration and authentication
- Proposed process: Product search, filtering and product-detail viewing
- Proposed process: Shopping-cart management and checkout
- Proposed process: Payment-status recording through a simulated or sandbox gateway

Deliverable 6: UML Design

Submit the following UML artefacts and ensure they agree with one another:

- One system use-case diagram with all major actors
- Detailed descriptions for at least six use cases
- At least two activity diagrams
- At least two sequence diagrams
- One domain class diagram with attributes, relationships and multiplicities

Suggested Use Cases

- Register or log in
- Browse and search products
- Manage shopping cart
- Checkout
- Record payment status
- Fulfil order
- Track order
- Request return
- Manage catalogue
- Generate reports

Use-case field	Required content
ID and name	Unique reference and action-oriented name
Primary actor	Role that starts the use case
Trigger	Event that starts the process
Preconditions	What must already be true
Normal flow	Numbered successful interaction
Alternative and exception flows	Expected variations and failures
Postconditions	Result and persistent data changes

Deliverable 7: Database and Data Design

Design a normalised relational database. Submit an entity-relationship diagram, table definitions, keys, constraints, relationship cardinalities and a data dictionary.

Suggested Entities

- User
- Role
- CustomerProfile
- Address
- Product
- Category
- InventoryItem
- Cart
- CartItem
- Order
- OrderItem
- PaymentRecord
- Shipment
- ReturnRequest
- Promotion
- AuditEntry

The final ERD must include at least ten related entities and must resolve any many-to-many relationships using appropriate linking entities.

Table	Field	Data type	Required	Key or rule	Description

Deliverable 8: User-Interface and Navigation Design

Create at least ten annotated wireframes. Show navigation, fields, labels, validation, success/error feedback, empty states, loading states, confirmation for destructive actions and responsive behaviour.

Suggested Screens

- Login and registration
- Home/catalogue page
- Product details
- Shopping cart
- Checkout
- Customer order history
- Order details/tracking
- Product administration
- Stock dashboard
- Fulfilment queue
- Returns dashboard
- Management reports

Design review question	Evidence expected
Can the user identify the next action?	Clear hierarchy and primary action
Can keyboard and assistive-technology users complete the task?	Labels, focus order, error association and keyboard access
Are sensitive fields protected?	Masking, minimum display and appropriate confirmation
Are exceptions understandable?	Specific validation and recovery guidance
Does the layout work on mobile and desktop?	Responsive variants or annotations

Deliverable 9: Algorithm and Logic Design

Develop pseudocode and a flowchart for at least three critical processes:

1. Validate stock and reserve items during checkout
2. Calculate subtotal, discounts, delivery charge and final total
3. Release or adjust stock after cancellation or approved return

```

START
INPUT request
VALIDATE required fields and user permission
IF validation fails THEN
    RETURN specific error
END IF
BEGIN TRANSACTION
    APPLY business rules
    SAVE changes and audit event
COMMIT TRANSACTION
RETURN success response
END

```

Deliverable 10: Technical Architecture

Use a responsive web client, an application API, a relational database, object storage for product images, a simulated payment adapter, notification service and audit logging. The student solution must never collect real card details.

Layer or component	Proposed technology	Reason and trade-off
User interface		
Application/API		
Database		
Authentication and authorisation		
Notifications		
File or media storage		
Audit and monitoring		
Deployment environment		

Technology selection: The qualification is vendor-neutral. Select technologies based on requirements, team capability, security, maintainability and deployment constraints rather than popularity alone.

Deliverable 11: Preliminary API Design

Propose at least twelve API operations. For each operation, document the request, response, validation errors, permissions, status codes and related user story.

Method	Endpoint	Purpose
POST	/api/auth/register	Register a customer
POST	/api/auth/login	Authenticate a user
GET	/api/products	Search active products
GET	/api/products/{id}	View product details
POST	/api/cart/items	Add an item to the cart
PUT	/api/cart/items/{id}	Update cart quantity
POST	/api/orders	Create an order
GET	/api/orders/{id}	View an authorised order
PATCH	/api/orders/{id}/status	Update fulfilment status
POST	/api/orders/{id}/returns	Request a return

Deliverable 12: Security, Privacy and Ethical Design

Prepare a threat and control register containing at least twelve risks. Address authentication, authorisation, validation, secure data handling, logging, backup, error handling, privacy, accessibility, ethical design and misuse prevention.

Risk	Impact	Proposed control
Account takeover	High	Secure password hashing, rate limiting and optional multi-factor authentication
Price manipulation in the browser	High	Calculate authoritative totals on the server
Overselling stock	High	Transactional stock validation and reservation
Injection or malicious input	High	Validation, parameterised queries and output encoding
Exposure of personal information	High	Least privilege, masking, encryption in transit and controlled logs
Fraudulent status changes	Medium	Role-based access and immutable audit entries
Loss of order data	High	Backups, restore testing and controlled deployment
Inaccessible user experience	Medium	Keyboard support, labels, contrast and responsive design

Mandatory safety rule: Use only sandbox payment services and fictional customer data. No real card numbers, identity numbers or confidential business data may be used.

Deliverable 13: Testing Strategy and Test Cases

Prepare a test strategy covering unit, integration, system, user acceptance, security, accessibility, performance and recovery testing. Write at least twenty test cases, including valid, invalid, boundary, permission, concurrency and recovery scenarios.

Test ID	Requirement	Scenario	Test data	Expected result

Deliverable 14: Requirements Traceability Matrix

Demonstrate that every high-priority requirement is represented in the design and can be verified by a test.

Requirement	User story	Use case	Design component	Test case

Deliverable 15: Four-Week Project Plan

Week	Focus	Minimum outputs
1	Discover and define	Scenario analysis, stakeholder map, research plan, charter and scope
2	Requirements and process modelling	SRS, user stories, backlog, current and proposed process models
3	Solution and data design	UML, ERD, data dictionary, algorithms, architecture and API design
4	Interface, security, testing and presentation	Wireframes, security register, test plan, traceability, final review and presentation

Deliverable 16: Teamwork and Version-Control Evidence

- Create a shared Git repository or controlled document workspace.

- Maintain meeting notes, task allocations, design decisions and review feedback.
- Use meaningful commit messages or version labels.
- Record individual contributions and peer feedback.
- Resolve design disagreements professionally and document the final decision.

Date	Decision or task	Owner	Evidence or version

Deliverable 17: Final Design Presentation

Deliver a 10-15 minute presentation covering the problem, users, scope, requirements, key processes, UML, data design, wireframes, architecture, security, tests, limitations and recommended implementation sequence. Every group member must participate.

Submission Structure

```

Project-Design-Portfolio/
├── 01-Project-Charter/
├── 02-Stakeholders-and-Research/
├── 03-Requirements-Specification/
├── 04-User-Stories-and-Backlog/
├── 05-Process-Models/
├── 06-UML-Diagrams/
├── 07-Database-Design/
├── 08-Wireframes-and-Navigation/
├── 09-Algorithms/
├── 10-Architecture-and-API/
├── 11-Security-and-Privacy/
├── 12-Testing-and-Traceability/
├── 13-Project-Evidence/
├── 14-Presentation/
└── README.md

```

The README must explain the project purpose, team members, tools used, folder contents, assumptions and how the approved design should progress into implementation.

Assessment Rubric

Assessment criterion	Marks
Problem definition, charter and scope	8
Stakeholders and requirements-gathering	7
Functional, non-functional and business requirements	15
User stories, acceptance criteria and backlog	10
Process models and UML design	15
Database and data dictionary	12
Wireframes, navigation and accessibility	10
Algorithms, architecture and API design	8

Assessment criterion	Marks
Security, privacy and ethics	6
Testing and traceability	5
Teamwork and version evidence	2
Presentation and professional quality	2
TOTAL	100

Performance Levels

Range	Descriptor
80-100%	Excellent: complete, consistent, traceable and professionally justified design with strong security, usability and testing evidence.
70-79%	Competent: workable and mostly consistent design that another team could implement with limited clarification.
50-69%	Developing competence: core components are present but contain gaps, weak traceability or insufficient technical detail.
Below 50%	Not yet competent: major requirements or models are missing, contradictory or cannot guide safe implementation.

Minimum Competence Requirements

- A clear problem statement, scope and stakeholder analysis.
- At least 25 functional and 10 non-functional requirements.
- At least 18 user stories and a 25-item prioritised backlog.
- A current-state process and at least three proposed process models.
- Use-case, activity, sequence and class diagrams.
- An ERD with at least ten entities and a complete data dictionary.
- At least ten annotated wireframes.
- Three algorithms and a high-level architecture.
- At least twelve API operations and twelve security/privacy risks.
- At least twenty test cases and a traceability matrix.
- Project evidence and a final presentation.

Academic Integrity and Responsible Use of AI

AI tools may support brainstorming, grammar improvement, diagram review, pseudocode explanation and test-idea generation. Learners remain responsible for accuracy, originality, privacy, source acknowledgement and understanding every submitted artefact. Generated content that the learner cannot explain does not demonstrate competence.

- Declare meaningful AI assistance.
- Do not paste confidential, personal or real operational information into AI tools.
- Verify generated requirements, code, diagrams and claims.
- Use fictional datasets that comply with the scenario safety note.
- Be prepared to explain and defend every design decision.

Facilitator Review Guide

Review point	Questions to ask learners
Requirements review	Which stakeholder problem does each high-priority requirement solve? Is the requirement testable?

Review point	Questions to ask learners
Process review	What happens when information is missing, invalid, duplicated or unauthorised?
Data review	Why is each entity required? Are keys, relationships and retention assumptions appropriate?
Interface review	Can users complete the task safely and accessibly? Are errors and sensitive information handled properly?
Architecture review	How does the design enforce permissions, transactions, audit and recovery?
Testing review	Does every high-priority requirement have positive, negative and permission-focused tests?

Preparation for the Development Phase

After the design portfolio is approved, learners may create an implementation backlog, repository, database migrations, application skeleton, authentication, core features, automated tests, deployment configuration and a demonstration environment. The approved SRS, models, wireframes, data design and traceability matrix must remain the baseline for controlled change.

Final submission check: Confirm that all documents use consistent IDs, terminology, roles, statuses, entities and business rules. Resolve contradictions before submission.